MECHATRONICS AND IOT

ASSIGNMENT - 8

Wireless Environmental Parameter Monitoring System

TEAM MEMBERS:

JEEVA T	**24MER020**
NITHEESH M	**24MER036**
SANJAY S	**24MER054**
SIVA BALAJI S	**24MER058**
SIVA S	**24MER059**

Wireless Environmental Parameter Monitoring System

1. Project Overview

The Wireless Environmental Parameter Monitoring System is an IoT-based system designed to continuously monitor important environmental conditions in industrial environments. The system measures parameters such as temperature, humidity, and air quality using suitable sensors and transmits the collected data wirelessly to a cloud platform.

An ESP8266 Node MCU is used as the main controller because it provides Wi-Fi connectivity and can process sensor data before transmitting it to the cloud. A DHT11/DHT22 sensor is used to measure temperature and humidity, while an MQ135 gas sensor is used to monitor air-quality conditions.

The collected environmental data is uploaded to the Adafruit IO cloud platform, where it can be stored, visualized, and monitored remotely. This system can help industries identify abnormal environmental conditions and support safer and more efficient industrial operations.

Main components

- Esp32
- DHT11 Sensor
- MQ135 Air Quality Sensor
- OLED Display
- Buzzer
- Breadboard
- Jumper Wires

2. Project Statement

Industrial environments require continuous monitoring of environmental conditions to maintain worker safety, equipment reliability, and proper operating conditions. Excessive temperature, high humidity, and poor air quality can affect industrial processes and may create unsafe working conditions.

Traditional monitoring methods may require manual inspection and may not provide continuous remote access to environmental data.

Awireless IoT-based monitoring system is proposed to measure temperature, humidity, and air quality in real time and transmit the measured parameters to a cloud platform for remote monitoring and analysis.

3. Proposed Solution

The proposed system uses multiple environmental sensors connected to an ESP8266 NodeMCU.

The DHT11/DHT22 sensor measures the surrounding temperature and humidity, while the MQ135 sensor detects changes in air quality. The ESP8266 collects the sensor readings, processes the data, and connects to the internet through Wi-Fi.

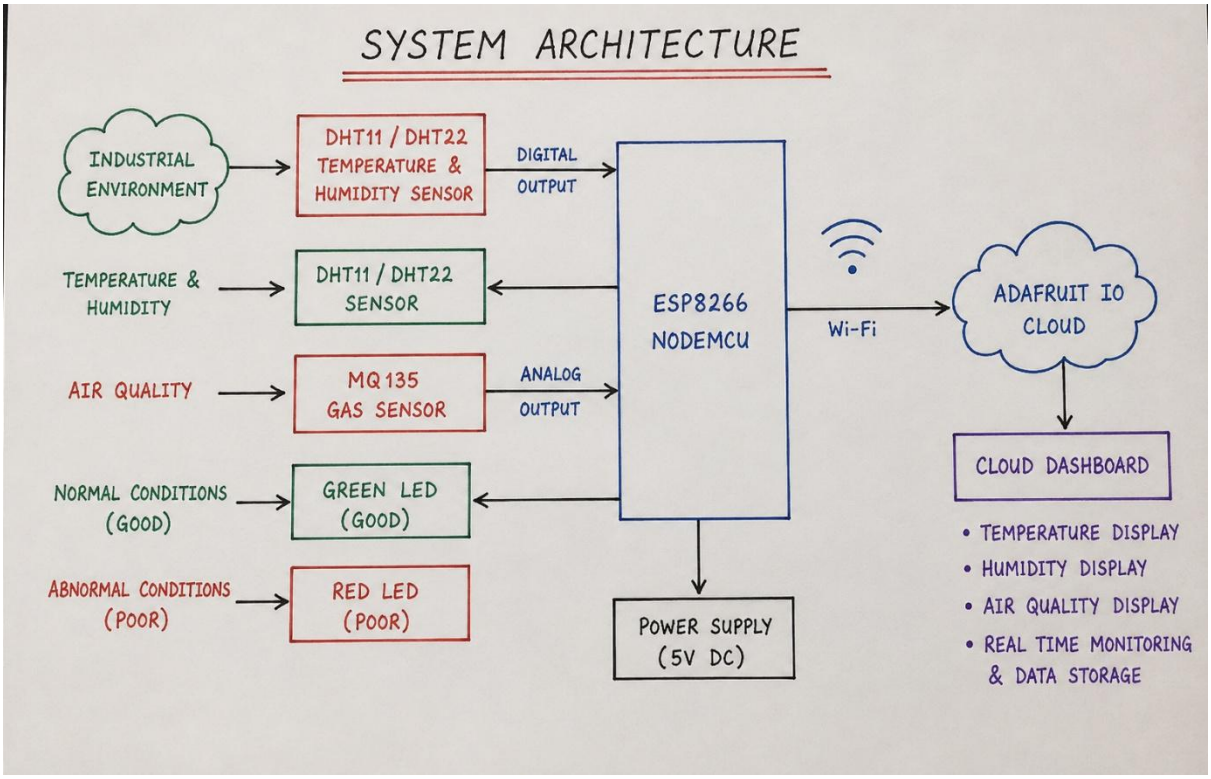
The measured parameters are then uploaded to Adafruit IO using wireless communication. The cloud dashboard provides a graphical representation of the environmental conditions and allows the user to monitor the industrial environment remotely.

Using its built-in Wi-Fi capability, the ESP8266 transmits the collected environmental data to the **Adafruit IO cloud platform**. Separate cloud feeds are used to store the temperature, humidity, and air-quality readings. The data can then be displayed on an Adafruit IO dashboard, allowing authorized users to remotely monitor the industrial environment in real time.

The system provides three major monitoring parameters:

1. Temperature monitoring
2. Humidity monitoring
3. Air-quality monitoring

4. System Architecture:



5. Circuit Table:

CIRCUIT TABLE

1. DHT11 / DHT22 TEMPERATURE & HUMIDITY SENSOR CONNECTION

Component	Pin	ESP8266 (NodeMCU) Pin	Function
DHT11 / DHT22 Sensor	VCC	3.3V	Power Supply (3.3V)
	DATA	D4 (GPIO2)	Digital Data Output
	GND	GND	Ground

2. MQ135 AIR QUALITY SENSOR CONNECTION

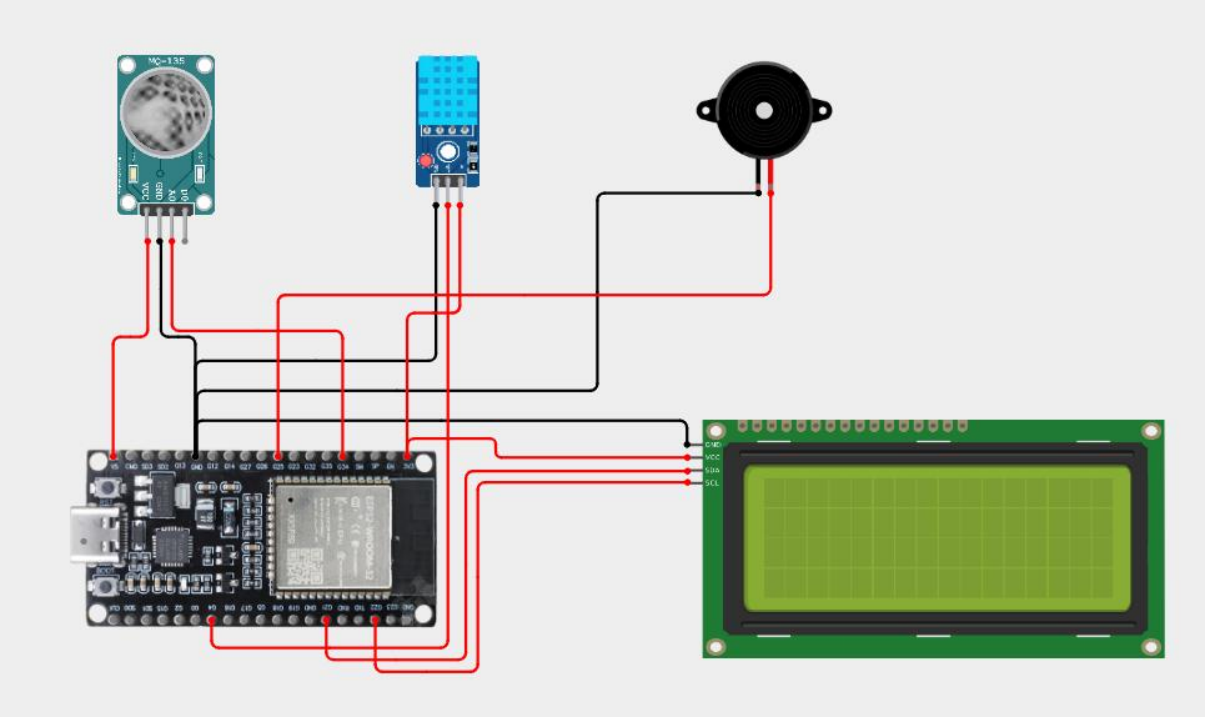
Component	Pin	*ESP8266 (NodeMCU) Pin*	Function
MQ135 Gas Sensor	VCC	VIN / 5V	Power Supply (5V)
	GND	GND	Ground
	AO	A0	Analog Output to ESP8266

3. POWER SUPPLY

Component	Pin	Connection	Function
ESP8266 NodeMCU	VIN	5V DC (USB / External 5V Supply)	Power Supply to ESP8266
ESP8266 NodeMCU	GND	Common Ground	Ground
All Components	GND	Common Ground with ESP8266	Ensure *Common Ground*

Note: Use common ground (GND) for ESP8266, DHT11/DHT22 and MQ135 sensor.

6. Circuit Design:



7. Algorithm:

The algorithm of the Wireless Environmental Parameter Monitoring System for Smart Industries defines the sequence of operations performed by the ESP8266 NodeMCU to collect, process, transmit, and monitor environmental parameters. The system continuously measures temperature, humidity, and air quality and uploads the measured data to the Adafruit IO cloud platform for remote monitoring.

- Start the system.
- Initialize the ESP8266 NodeMCU.
- Initialize the DHT sensor.
- Initialize the MQ135 sensor.
- Connect the ESP8266 to the Wi-Fi network.
- Establish a connection with Adafruit IO.
- Read temperature from the DHT sensor.
- Read humidity from the DHT sensor.
- Read the analog air-quality value from the MQ135.
- Process the collected sensor values.
- Send temperature data to the cloud.
- Send humidity data to the cloud.
- Send air-quality data to the cloud.
- Display the parameters on the Adafruit IO dashboard.
- Wait for a predefined time interval.
- Repeat the monitoring process continuously.

8. Coding:

```
#include "AdafruitIO_WiFi.h"

#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#include <DHT.h>
```

```
// ============================================================
// WIFI DETAILS
// ============================================================

#define WIFI_SSID       "OnePlus Nord CE5 7C5C"
#define WIFI_PASSWORD   "qkts5737"


// ============================================================
// ADAFRUIT IO DETAILS
// ============================================================

#define IO_USERNAME     "Hair_13"
#define IO_KEY          "aio_EMRC40vci7DQ5Amz5kvOKh6WKHI0"


// Create Adafruit IO WiFi object
AdafruitIO_WiFi io(
  IO_USERNAME,
  IO_KEY,
  WIFI_SSID,
  WIFI_PASSWORD
);


// ============================================================
// FEEDS
// ============================================================

AdafruitIO_Feed *temperatureFeed =
  io.feed("temperature");

AdafruitIO_Feed *humidityFeed =
  io.feed("humidity");

AdafruitIO_Feed *mq135Feed =
  io.feed("mq135");

AdafruitIO_Feed *airQualityAlertFeed =
  io.feed("air-quality-alert");


// ============================================================
// DHT11
// ============================================================

#define DHT_PIN 4
#define DHT_TYPE DHT11
```

```
DHT dht(DHT_PIN, DHT_TYPE);


// ========================================================
// MQ135 AIR QUALITY SENSOR
// ========================================================

#define MQ135_PIN 34


// ========================================================
// BUZZER
// ========================================================

#define BUZZER_PIN 25


// ========================================================
// OLED
// ========================================================

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

Adafruit_SSD1306 display(
  SCREEN_WIDTH,
  SCREEN_HEIGHT,
  &Wire,
  -1
);


// ========================================================
// VARIABLES
// ========================================================

float temperature = 0;
float humidity = 0;

int mq135Value = 0;

bool airQualityAlert = false;


// ========================================================
// TIMING
// ========================================================

unsigned long lastSend = 0;
```

```
const unsigned long sendInterval =
  15000;


// ========================================================
// SETUP
// ========================================================

void setup()
{

  Serial.begin(9600);

  delay(1000);


  // ------------------------------------------------
  // DHT
  // ------------------------------------------------

  dht.begin();


  // ------------------------------------------------
  // SENSOR PINS
  // ------------------------------------------------

  pinMode(MQ135_PIN, INPUT);

  pinMode(BUZZER_PIN, OUTPUT);

  digitalWrite(
    BUZZER_PIN,
    LOW
  );


  // ------------------------------------------------
  // OLED
  // ------------------------------------------------

  Wire.begin(21, 22);

  if (!display.begin(
        SSD1306_SWITCHCAPVCC,
        0x3C))
  {
```

```
  Serial.println(
    "OLED not detected!"
  );
}
else
{

  display.clearDisplay();

  display.setTextColor(
    SSD1306_WHITE
  );

  display.setTextSize(1);

  display.setCursor(10, 10);

  display.println(
    "SMART MONITOR"
  );

  display.setCursor(10, 30);

  display.println(
    "Starting..."
  );

  display.display();
}


// ------------------------------------------------
// CONNECT TO ADAFRUIT IO
// ------------------------------------------------

Serial.println();
Serial.println(
  "Connecting to Adafruit IO..."
);

io.connect();


// Wait for connection
while (io.status() < AIO_CONNECTED)
{

  Serial.print(".");
```

```
    delay(500);
  }


  Serial.println();

  Serial.println(
    "Connected to Adafruit IO!"
  );


  display.clearDisplay();

  display.setTextSize(1);

  display.setCursor(0, 10);

  display.println(
    "Adafruit IO"
  );

  display.setCursor(0, 30);

  display.println(
    "Connected!"
  );

  display.display();

  delay(2000);
}


// =================================================
// LOOP
// =================================================

void loop()
{

  // Keep Adafruit IO connection alive
  io.run();


  // ------------------------------------------------
  // SEND SENSOR DATA
  // ------------------------------------------------

  if (
```

```
  millis() - lastSend >=
  sendInterval
)
{

  lastSend = millis();


  // ================================================
  // DHT11
  // ================================================

  temperature =
    dht.readTemperature();

  humidity =
    dht.readHumidity();


  if (isnan(temperature) ||
      isnan(humidity))
  {

    Serial.println(
      "DHT11 reading failed!"
    );

  }
  else
  {

    Serial.print(
      "Temperature: "
    );

    Serial.print(
      temperature
    );

    Serial.println(" C");


    Serial.print(
      "Humidity: "
    );

    Serial.print(
      humidity
    );
```

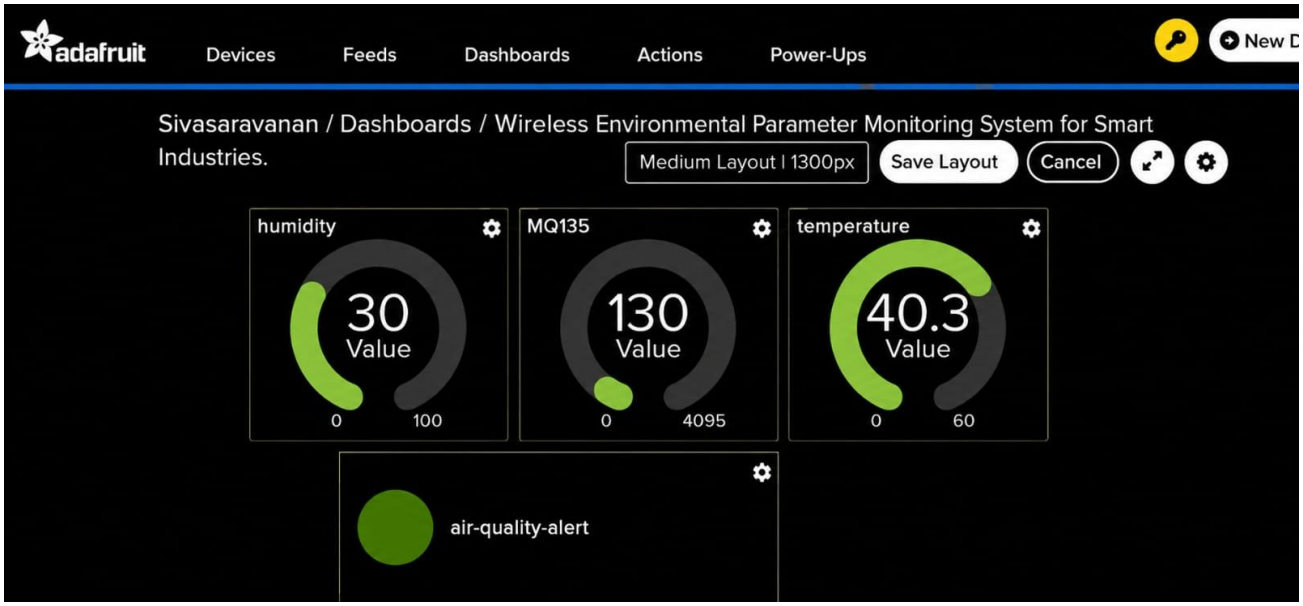
```
  Serial.println(" %");
}


// ================================================
// MQ135
// ================================================

mq135Value =
  analogRead(MQ135_PIN);


Serial.print(
  "MQ135: "
);

Serial.println(
  mq135Value
);


// ================================================
// AIR QUALITY ALERT
// ================================================

// Adjust this threshold after testing

if (mq135Value > 2000)
{

  airQualityAlert = true;

  digitalWrite(
    BUZZER_PIN,
    HIGH
  );

}
else
{

  airQualityAlert = false;

  digitalWrite(
    BUZZER_PIN,
    LOW
  );
}
```

```
// ================================================
// SEND TO ADAFRUIT IO
// ================================================

Serial.println();

Serial.println(
  "Sending data to Adafruit IO..."
);


if (!isnan(temperature))
{

  temperatureFeed->save(
    temperature
  );
}


if (!isnan(humidity))
{

  humidityFeed->save(
    humidity
  );
}


mq135Feed->save(
  mq135Value
);


airQualityAlertFeed->save(
  airQualityAlert ? 1 : 0
);


Serial.println(
  "Data sent successfully!"
);


// ================================================
// OLED
// ================================================
```

```
display.clearDisplay();

display.setTextColor(
  SSD1306_WHITE
);

display.setTextSize(1);


display.setCursor(0, 0);

display.print("T:");

display.print(
  temperature,
  1
);

display.print("C ");


display.print("H:");

display.print(
  humidity,
  0
);

display.println("%");


display.setCursor(0, 15);

display.print("MQ135: ");

display.println(
  mq135Value
);


display.setCursor(0, 30);

if (airQualityAlert)
{

  display.println(
    "AIR QUALITY ALERT!"
  );
```

```
    }
    else
    {

      display.println(
        "Air Quality Normal"
      );
    }


    display.setCursor(0, 45);

    display.print("WiFi: ");

    display.println(
      "Connected"
    );


    display.display();


    Serial.println(
      "================================="
    );
  }
}
```

9. System Working :



When the system is powered ON, the ESP8266 initializes all connected sensors and attempts to connect to the available Wi-Fi network.

After establishing the internet connection, the ESP8266 connects to the Adafruit IO cloud platform. The DHT11/DHT22 sensor continuously measures the temperature and humidity of the industrial environment.

At the same time, the MQ135 sensor detects changes in the surrounding air and provides an analog output to the ESP8266.

The ESP8266 reads all the sensor values and sends them to the corresponding cloud feeds.

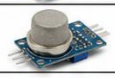
The system therefore provides:

- Real-time temperature monitoring
- Real-time humidity monitoring
- Air-quality monitoring
- Wireless data transmission
- Cloud-based data storage
- Remote monitoring through a dashboard

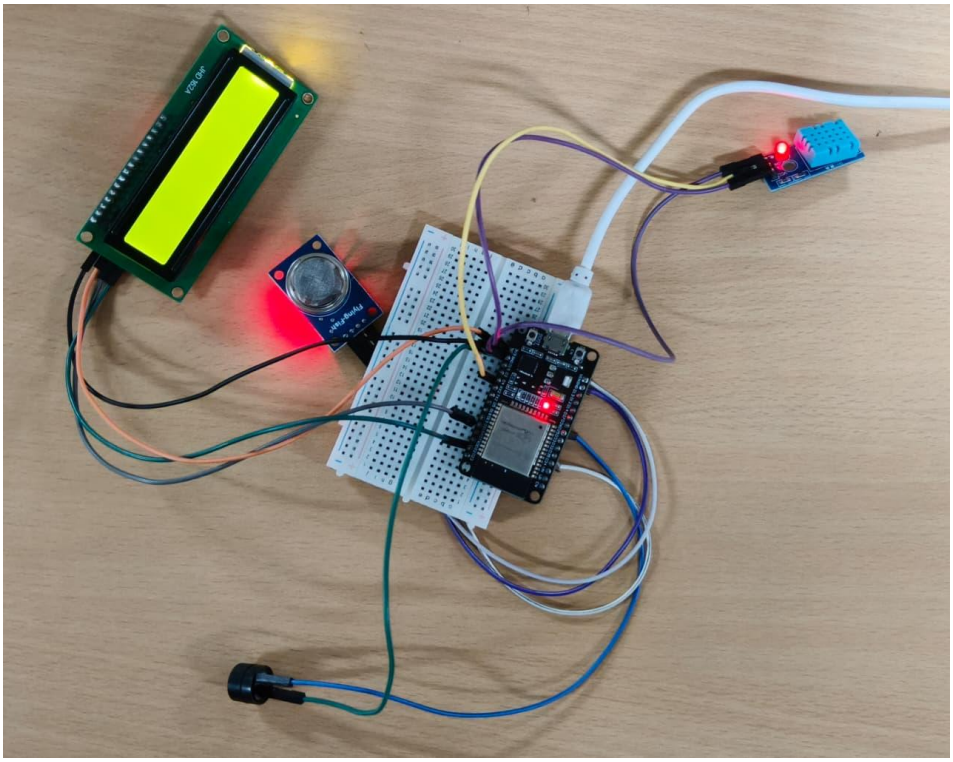
The user can access the dashboard remotely and observe changes in the environmental conditions.

10. BILL OF MATERIAL:

BILL OF MATERIAL

S.No.	Component		Specification / Description	Quantity
1.		**ESP8266 NodeMCU**	ESP8266 Wi-Fi Development Board	1
2.		**DHT11 / DHT22** Temperature & Humidity Sensor	Measures Temperature (°C) and Relative Humidity (%)	1
3.		**MQ135 Gas Sensor**	Air Quality / Gas Sensing Module (Analog Output)	1
4.		**Green LED**	5mm LED (Indicates Good Air Quality)	1
5.		**Red LED**	5mm LED (Indicates Poor Air Quality)	1
6.		**Resistor**	220 Ω Resistor (for LEDs)	2
7.		**Breadboard**	Solderless Breadboard	1
8.		**Jumper Wires**	Male to Male Jumper Wires	As required
9.		**USB Cable**	Micro USB Cable for Power & Programming	1
10.		**Wi-Fi Connection**	Internet Connection for ESP8266	1
		**Adafruit IO Cloud**	Cloud Platform for Data Monitoring and Visualization	1

11. Prototype Implementation:



The prototype was implemented using an ESP8266 NodeMCU, DHT11/DHT22 temperature and humidity sensor, and MQ135 air-quality sensor.

The sensors were connected to the ESP8266 according to the designed circuit. The ESP8266 was programmed using the Arduino IDE to collect environmental data at regular intervals.

A Wi-Fi connection was established to provide wireless communication between the prototype and the Adafruit IO cloud platform.

Separate cloud feeds were created for:

- Temperature
- Humidity
- Air Quality

The sensor data was successfully transmitted to the cloud, where it could be monitored through a dashboard. The prototype demonstrates how wireless sensing and cloud technology can be integrated for environmental monitoring in smart industrial applications.

12. Results and Performance Analysis:

The developed Wireless Environmental Parameter Monitoring System for Smart Industries was successfully implemented and tested for monitoring important environmental parameters such as temperature, humidity, and air quality. The system continuously collects data from the connected sensors and processes the readings using the ESP32 microcontroller. The measured values can be displayed locally and transmitted wirelessly, allowing the user to monitor the industrial environment remotely. The prototype demonstrated that the system can provide continuous environmental information with minimum human intervention.

During testing, the DHT11/DHT22 sensor successfully measured the surrounding temperature and relative humidity. The temperature readings changed according to the surrounding environmental conditions, while the humidity values responded to changes in moisture levels. The sensor provided stable readings during normal operating conditions and was suitable for demonstrating basic environmental monitoring in the prototype.

The MQ-135 air-quality sensor was used to detect changes in the surrounding air conditions. When the sensor was exposed to relatively clean air, the sensor output remained comparatively low and stable. When smoke, gases, or other pollutants were introduced near the sensor, the output value increased. This demonstrated that the sensor can be used for detecting changes in air quality. However, the MQ-135 requires proper calibration if accurate gas concentration values such as ppm are required.

Overall, the experimental results show that the proposed system is a low-cost, compact, and practical solution for basic environmental monitoring in smart industrial applications. With improved sensor calibration, industrial-grade sensors, reliable cloud connectivity, data logging, and advanced alert mechanisms, the prototype can be further developed into a more accurate and reliable industrial environmental monitoring system.